

Arrays

1. Definition

A data structure that stores a fixed number of elements of the same type, accessible by an index.

Key Points

- **Same type:** All elements must be of the same data type (e.g., all `int`, all `String`).
- **Fixed size:** Once created, the length of the array cannot change.
- **Indexed:** Each element is identified by a position number (index), starting at 0.
- **Contiguous memory:** Elements are stored next to each other in memory.

```
int[] numbers = {10, 20, 30, 40};
System.out.println(numbers[0]); // prints 10
System.out.println(numbers[3]); // prints 40
```

2. Declaration & Creation

Declare an array variable only (no size, no values yet):

```
int[] numbers;    // Preferred style
int numbers[];    // Also valid (C-style)
```

Declare an array + create array object(size known, default values assigned):

```
int[] numbers = new int[5];
// Creates array of 5 integers, default values = 0
```

Default values in arrays

Data Type	Default Value
byte	0
int	0
long	0L
float	0.0f
double	0.0d
char	'\u0000' (null character)
boolean	false
Reference types (e.g., String, objects)	null

Declare an array + create array object + init values:

```
int[] numbers = {10, 20, 30, 40, 50};
String[] cars = {"Volvo", "BMW", "Ford"};
```

Two-step (declare first, assign later):

```
int[] numbers;
numbers = new int[]{10, 20, 30, 40, 50};
```

Ways to Create Arrays

Way	Syntax	Notes
With values	<code>int[] a = {1, 2, 3};</code> or <code>int[] a = new int[]{1, 2, 3};</code> ✓	Use when you know values
With size only	<code>int[] a = new int[3];</code> ✓	Creates array with default values (0 for int)
Size + values	<code>int[] a = new int[3]{1,2,3};</code> ✗	Not allowed in Java

3. Access & Assign

```
int[] nums = new int[3];
nums[0] = 10;
System.out.println(nums[0]); // prints 10
```

4. .length Property

- `.length` gives the size of the array.
- **Note:** `.length` is a **property**, not a method.

~~`.length()`~~ ✗

```
int [] myNum = {10, 20, 30, 40};
System.out.println(myNum.length); // prints 4
```

5. Looping Through Arrays

Basic **for** loop

Uses index to access elements.

```
int[] numbers = {10, 20, 30, 40};

for (int i = 0; i < numbers.length; i++) {
    System.out.println("Index " + i + ": " + numbers[i]);
}
```

Enhanced **for-each** loop

Shorter, no index, but cannot change array elements directly.

```
for (int num : numbers) {
    System.out.println(num);
}
```

- ✓ Use **basic for loop** if you need index or want to update elements.
 - ✓ Use **enhanced for-each** for simple read-only traversal.
-

6. Shallow Copy vs Deep Copy

Shallow Copy

- Copies **reference only**.

- Both variables point to the **same array**.

```
int[] a = {1, 2, 3};
int[] b = a; // shallow copy
b[0] = 99;

System.out.println(a[0]); // 99
```

Deep Copy

- Creates a **new array** and copies all values.
- Arrays are **independent**.

```
int[] a = {1, 2, 3};
int[] b = new int[a.length];

for (int i = 0; i < a.length; i++) {
    b[i] = a[i];
}

b[0] = 99;

System.out.println(a[0]); // 1
System.out.println(b[0]); // 99
```

7. Arrays Utility Class (`java.util.Arrays`)

Method	Description	Example
<code>Arrays.fill(array, value)</code>	Fill array with value	<code>Arrays.fill(arr, 1);</code>
<code>Arrays.sort(array)</code>	Sort array ascending	<code>Arrays.sort(arr);</code>
<code>Arrays.copyOf(array, newLength)</code>	Copy array with new size	<code>int[] newArr = Arrays.copyOf(arr, 10);</code>
<code>Arrays.equals(arr1, arr2)</code>	Compare 1D arrays	<code>Arrays.equals(arr1, arr2);</code>
<code>Arrays.toString(array)</code>	Convert 1D array to string	<code>System.out.println(Arrays.toString(arr));</code>
<code>Arrays.deepEquals(arr1, arr2)</code>	Compare multidimensional arrays	<code>Arrays.deepEquals(matrix1, matrix2);</code>
<code>Arrays.deepToString(array)</code>	Convert multidimensional array to string	<code>System.out.println(Arrays.deepToString(matrix));</code>